

Utilização *de strings*

Strings são sequências de caracteres, e boa parte do trabalho de um programa é lidar com texto: nomes, mensagens, arquivos, respostas de usuários. Nesta aula: os jeitos de escrever uma string, caracteres especiais, índices positivos e negativos, fatias, por que strings não podem ser alteradas e como percorrer letra por letra.

' " ""

\n · \t · r"

Índices

Fatias [i:j:passo]

len()

Imutável

in

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Text; Built-in Types: Text Sequence Type; Lexical analysis: string literals), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Aspas simples, duplas *e triplas*

criando.py

```
simples = 'texto com aspas simples'
duplas = "texto com aspas duplas"
com_apostrofo = "Copo d'água"  # duplas fora, simples dentro
com_aspas = 'Ele disse "oi"'    # e vice-versa
varias_linhas = """Linha 1
Linha 2"""
print(simples)
print(com_apostrofo)
print(com_aspas)
print(varias_linhas)
```

SAÍDA

```
texto com aspas simples
Copo d'água
Ele disse "oi"
Linha 1
Linha 2
```

- ✓ `'...'` e `"..."` são equivalentes
- ✓ Escolha a que **não** aparece no texto: "Copo d'água"
- ✓ `"""..."""` aceita várias linhas
- ✓ Escolha um padrão e use sempre o mesmo no projeto

Caracteres *de escape*

escapes.py

```
print("Coluna1\tColuna2")      # \t = tabulação
print("Linha 1\nLinha 2")     # \n = quebra de linha
print("Ele disse \"oi\"")      # \" = aspas dentro de aspas
print("C:\\pasta\\arquivo.txt") # \\ = uma barra invertida
print(r"C:\pasta\novo.txt")    # r"...": string crua
```

SAÍDA

```
Coluna1 Coluna2
Linha 1
Linha 2
Ele disse "oi"
C:\pasta\arquivo.txt
C:\pasta\novo.txt
```

ESCAPE	SIGNIFICA
\n	Quebra de linha
\t	Tabulação
\" e \'	Aspas dentro de aspas iguais
\\	Uma barra invertida

Caminhos do Windows têm \ : em "C:\novo" , o \n vira quebra de linha! Use r"..." (string crua) ou barras normais: "C:/novo" .

Cada caractere *tem uma posição*

indices.py

```
palavra = "Python"
#       P y t h o n
# índice: 0 1 2 3 4 5
# negativo:-6 ... -2 -1
print(palavra[0])    # P
print(palavra[5])    # n
print(palavra[-1])   # n (último)
print(palavra[-2])   # o
print(len(palavra))  # 6 caracteres
```

SAÍDA

```
P
n
n
o
6
```

LETRA	P	Y	T	H	O	N
Índice	0	1	2	3	4	5
Negativo	-6	-5	-4	-3	-2	-1

- ✓ A contagem começa em 0
- ✓ O último é `len(s) - 1`, ou simplesmente `-1`
- ✓ Índices negativos contam do fim

Fatias: *[início:fim:passo]*

fatias.py

```
palavra = "Python"
print(palavra[0:2])    # Py   (do 0 até antes do 2)
print(palavra[2:])     # thon (do 2 até o fim)
print(palavra[:4])     # Pyth (do começo até antes do 4)
print(palavra[:2])     # Pto  (de 2 em 2)
print(palavra[::-1])   # nohtyP (invertida)
print(palavra[1:100])  # ython (fatia não dá erro)
```

SAÍDA

```
Py
thon
Pyth
Pto
nohtyP
ython
```

- ✓ O início entra, o fim não: `[0:2]` pega as posições 0 e 1
- ✓ Sem início: desde o começo. Sem fim: até o final
- ✓ Passo: de quanto em quanto; negativo anda de trás para frente
- ✓ `::-1` inverte o texto

As mesmas regras de índices e fatias valem para **listas** e **tuplas** (aulas 09 e 10). Aprende uma vez, usa três.

Fora do texto: *IndexError*

erro_indice.py

```
palavra = "Python"  
print(palavra[6])
```

```
Traceback (most recent call last):  
  File "erro_indice.py", line 2, in <module>  
    print(palavra[6])  
          ~~~~~^  
IndexError: string index out of range
```

- ✓ "Python" tem 6 letras: índices de 0 a 5
- ✓ Pedir o índice 6 é pedir uma letra que não existe
- ✓ Fatias são tolerantes: `palavra[1:100]` não dá erro (seção 04)

Clássico erro "de um a mais": achar que o último índice é igual ao tamanho. O último é **tamanho – 1**.

Strings *não mudam*

imutavel.py · TENTAR TROCAR UMA LETRA

```
palavra = "Python"
palavra[0] = "J"
```

```
Traceback (most recent call last):
  File "imutavel.py", line 2, in <module>
    palavra[0] = "J"
    ~~~~~^
TypeError: 'str' object does not support item assignment
```

imutavel_ok.py · CRIAR UMA NOVA

```
palavra = "Python"
nova = "J" + palavra[1:] # cria uma string nova
print(nova)
print(palavra)          # a original continua igual
```

```
Jython
Python
```

Strings são **imutáveis**: toda "alteração" (inclusive os métodos da aula 08) cria uma string **nova**. Por isso é preciso guardar o resultado numa variável.

Letra por letra, *in e **

percorrer.py

```
for letra in "Ana":  
    print(letra)  
print("n" in "Ana")    # True  
print("A" * 3)         # AAA
```

SAÍDA

```
A  
n  
a  
True  
AAA
```

- ✓ `for letra in texto:` passa por cada caractere, em ordem
- ✓ O `for` é o assunto da **aula 14**: aqui é só uma prévia
- ✓ `in` procura um trecho dentro do texto
- ✓ `len()` conta os caracteres, inclusive espaços

Fatiando *um nome*

Enunciado: peça um nome completo e mostre a primeira e a última letra, as três primeiras, o nome ao contrário e o tamanho.

Uma resposta possível

```
# Exercício: fatiando um nome
nome = input("Nome completo: ").strip()
print("Primeira letra:", nome[0])
print("Última letra:", nome[-1])
print("Três primeiras:", nome[:3])
print("Ao contrário:", nome[::-1])
print("Tamanho:", len(nome))
```

SAÍDA (DIGITOU "CARLA DIAS")

```
Nome completo: Carla Dias
Primeira letra: C
Última letra: s
Três primeiras: Car
Ao contrário: saiD alraC
Tamanho: 10
```

- ✓ O espaço também conta no tamanho: 10
- ✓ Desafio: mostre só o primeiro nome (dica: `.find(" ")`, aula 08)

Teste o que *ficou*

1. Em `s = "Python"`, quanto vale `s[-1]`?

A) P

B) n

C) o

D) Erro

B. -1 é o último.

2. Quanto vale `"Python"[1:4]`?

A) "Pyt"

B) "yth"

C) "ytho"

D) "yt"

B. Do 1 até antes do 4.

3. O que acontece em `s[0] = "J"`?

A) Troca a letra

B) `TypeError`: strings são imutáveis

C) `IndexError`

D) Nada

B. Crie uma nova: `"J" + s[1:]`.

4. Como escrever o caminho `C:\novo` sem problema?

A) `"C:\novo"`

B) `r"C:\novo"`

C) `'C:\novo'`

D) `""C:\novo""`

B. String crua: `\n` não vira quebra.

Texto é uma *sequência*

Strings se escrevem com aspas simples, duplas ou triplas, usam escapes como `\n` e `\t` (ou `r"..."` para não usar). Cada caractere tem índice a partir de **0**, fatias seguem `[início:fim:passo]` com o fim de fora, e strings são **imutáveis**.

Índice 0 e -1

[i:j:passo]

[::-1]

IndexError

Imutável

Próxima aula → 08 · Operações e Métodos em Strings

Fontes: docs.python.org/3/tutorial/introduction.html#text; docs.python.org/3/library/stdtypes.html#text-sequence-type-str; docs.python.org/3/reference/lexical_analysis.html#string-and-bytes-literals, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.